```
        doGenerate: async () => ({
          rawCall: { rawPrompt: null, rawSettings: {} },
          finishReason: 'stop',
          usage: { promptTokens: 10, completionTokens: 20 },
          text: `Hello, world!`,
        }),
      }),
      prompt: 'Hello, test!',
    });

    expect(result.text).toBe('Hello, world!');
  });
});
```

Defines the behavior of the mock model when generating text

Asserts that the returned text matches the expected output

The `MockLanguageModelV1` is instantiated to simulate a language model's behavior, where its `doGenerate` method is overridden to return a controlled response without making live calls. To test this code, you must first navigate to the ch08/chat-testing directory. Then, you can simply run the provided npm task named `test:vercel:ai`:

```
$ cd ch08/chat-testing
$ npm run test:vercel:ai


> chat-testing@0.0.0 test:vercel:ai
> jest --testPathPattern=vercel-ai

PASS  __tests__/test-vercel-ai.js
  Text Generation Tests
    ✓ should return predefined text from mock model (2 ms)
    ✓ should return a predefined object from mock model (2 ms)
```

Just as we used `generateText` to test strings, we can use `generateObject` with a schema and verify the output using `expect(result.object).toEqual()` by reviewing the implementation details in the same test file.

SIMULATEREADABLESTREAM

The helper function `simulateReadableStream` simulates streaming responses and is useful for testing the `streamObject` or `streamText` versions of the SDK. It basically enables developers to mimic the behavior of live data streams in a controlled environment.

To test streaming, you need to use both `MockLanguageModelV1` to create a mock instance of a model and then the `simulateReadableStream` to return a simulated stream of text in chunks. The following example code snippet shows how to define this instance:

```
const result = streamText({
      model: new MockLanguageModelV1({
        doStream: async () => ({
          stream: simulateReadableStream({
            chunks: [
              { type: 'text-delta', textDelta: 'This is ' },
```